

LangChain V1: Comprehensive Guide

Architecture, Components, and Code Implementations

Based on Jupyter Notebook Analysis

June 11, 2026

Contents

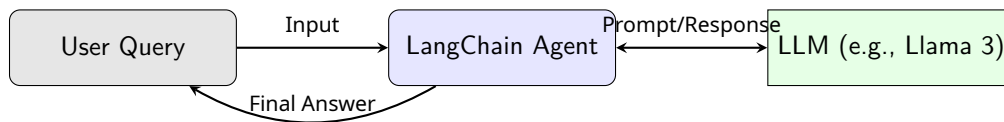
1	LangChain Introduction & Agents	2
1.1	Overview	2
1.2	Diagram: Basic Agent Flow	2
1.3	Code Snippet	2
2	Model Integration	2
2.1	Overview	2
2.2	Diagram: Model Abstraction	3
2.3	Code Snippet	3
3	Tools	4
3.1	Overview	4
3.2	Diagram: Tool Calling Mechanism	4
3.3	Code Snippet	4
4	Messages	4
4.1	Overview	4
4.2	Diagram: Message Stack	5
4.3	Code Snippet	5
5	Structured Output	6
5.1	Overview	6
5.2	Diagram: Structured Output Flow	6
5.3	Code Snippet	6
6	Middleware	6
6.1	Overview	6
6.2	Diagram: Middleware & Interrupts	7
6.3	Code Snippet	7

1 LangChain Introduction & Agents

1.1 Overview

LangChain is a framework designed to simplify the creation of applications using large language models (LLMs). The core idea is to string together different components to create advanced AI agents capable of reasoning and acting. In the V1 architecture, agents are initialized and invoked to process user queries.

1.2 Diagram: Basic Agent Flow



1.3 Code Snippet

To test a basic agent, we invoke it with a message structure. We also see the importance of loading environment variables (like API keys) using `dotenv`.

```
1 import os
2 from dotenv import load_dotenv
3
4 load_dotenv() # Load environment variables (.env)
5
6 # Invoking the agent with a basic message payload
7 response = agent.invoke({
8     "messages": [
9         {"role": "user", "content": "What is the weather in New York?"}
10    ]
11 })
12 print(response)
```

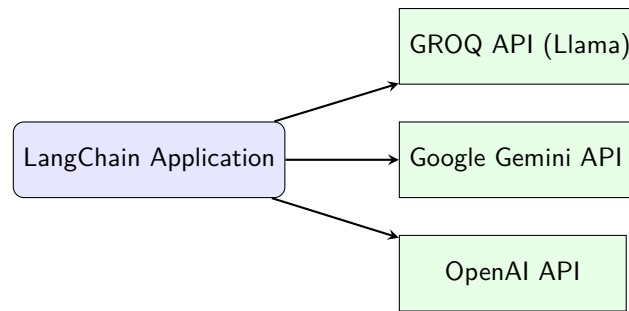
Listing 1: Basic Agent Invocation

2 Model Integration

2.1 Overview

LangChain allows seamless integration with various LLM providers, such as OpenAI, Google Gemini, and GROQ. By abstracting the model interfaces, you can easily switch between different models or perform batch processing to send multiple queries simultaneously, optimizing execution time.

2.2 Diagram: Model Abstraction



2.3 Code Snippet

Batching is highly efficient when you have independent queries that can be processed in parallel by the model.

```
1 import os
2 from langchain_groq import ChatGroq
3
4 # Setup GROQ integration
5 groq_api_key = os.getenv("GROQ_API_KEY")
6 model = ChatGroq(model="llama-3.1-8b-instant", api_key=groq_api_key)
7
8 # Batch processing multiple questions
9 response = model.batch([
10     "Why do parrots talk?",
11     "How much water does an elephant drink in a day?"
12 ])
13
14 for resp in response:
15     print(resp.content)
```

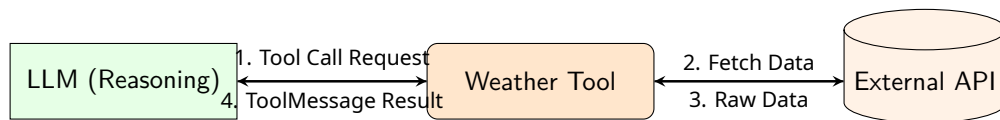
Listing 2: Model Batch Processing

3 Tools

3.1 Overview

Models are naturally limited to their training data. **Tools** provide a way for models to connect to the outside world. When a model realizes it needs external information (like current weather, database records, or web searches), it requests to call a Tool.

3.2 Diagram: Tool Calling Mechanism



3.3 Code Snippet

When a tool is invoked, the LLM generates a `tool_calls` dictionary, and the framework executes it, returning a `ToolMessage`.

```
1 # The model identifies the need for a tool and outputs a tool call:
2 # tool_calls=[{'name': 'get_weather', 'args': {'location': 'San Francisco'}, 'type': '
  tool_call'}]
3
4 # The execution returns a ToolMessage to the model:
5 # ToolMessage(content="It's sunny in San Francisco", name='get_weather', tool_call_id='
  fc85df7a...')
```

Listing 3: Tool Interaction Output

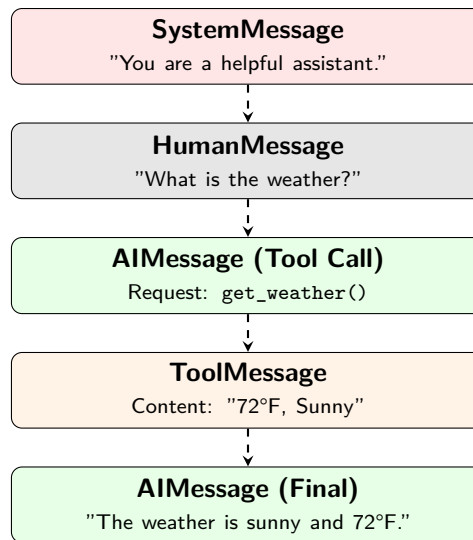
4 Messages

4.1 Overview

Messages are the fundamental unit of context in LangChain. They carry the conversational state. Every message contains:

- **Role:** Identifies who sent the message (System, User, AI, Tool).
- **Content:** The actual text, image, or structured data.
- **Metadata:** Tokens used, model name, finish reason, etc.

4.2 Diagram: Message Stack



4.3 Code Snippet

```
1 response = AIMessage(  
2     content="The weather in San Francisco is sunny with a temperature of 72°F.",  
3     response_metadata={  
4         'finish_reason': 'STOP',  
5         'model_name': 'gemini-3-flash-preview'  
6     },  
7     usage_metadata={'input_tokens': 105, 'output_tokens': 18}  
8 )
```

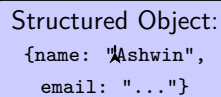
Listing 4: Inspecting an AIMessage

5 Structured Output

5.1 Overview

Often, you need an LLM to output data in a very specific format (like JSON) so your application can parse it programmatically. LangChain solves this by allowing you to define a schema using **Pydantic** or standard Python **dataclasses**. The LLM is forced to conform to this exact schema.

5.2 Diagram: Structured Output Flow



```
Structured Object:
{
  name: "Ashwin",
  email: "..."}

```

5.3 Code Snippet

```
1 from dataclasses import dataclass, Field
2 from langchain.agents import create_agent
3
4 @dataclass
5 class ContactInfo:
6     """Call information of the person"""
7     name: str = Field(description="The name of the person")
8     email: str = Field(description="The email address")
9     phone: str = Field(description="The phone number")
10
11 # The agent auto-selects and enforces the structure
12 agent = create_agent(
13     model=model,
14     response_format=ContactInfo
15 )
16
17 result = agent.invoke({
18     "messages": [{"role": "user", "content": "Extract contact info from: Ashwin mehta, ashwinmehta1234500@gmail.com, 888444676"}]
19 })
20
21 print(result["structured_response"])
```

Listing 5: Enforcing Output with Dataclasses

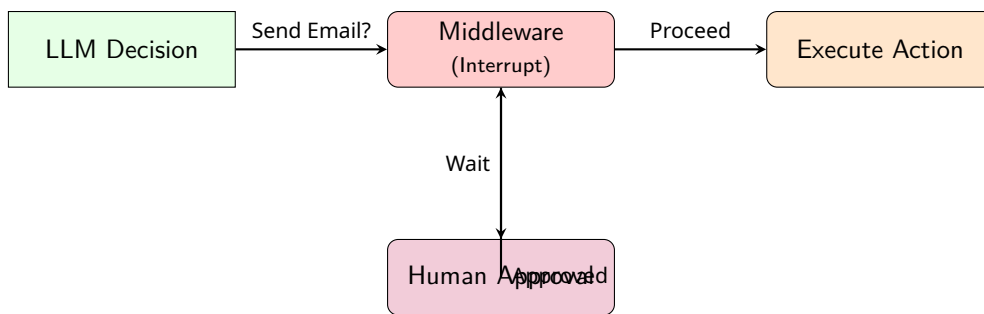
6 Middleware

6.1 Overview

Middleware in LangChain acts as an interception layer between components. It allows developers to tightly control the execution flow. Key uses include:

- Tracking agent behavior (logging, debugging).
- Applying rate limits.
- Adding retries and fallback logic.
- **Human-in-the-loop (Interrupts):** Pausing execution to ask a human to approve a critical tool action (e.g., sending an email).

6.2 Diagram: Middleware & Interrupts



6.3 Code Snippet

```
1 # The agent execution pauses and returns an Interrupt state
2 # requiring human approval for a specific tool call.
3 result = {
4     '__interrupt__': [{
5         'action_requests': [{
6             'name': 'send_email_tool',
7             'args': {'body': 'Hello', 'recipient': 'wrong@email.com', 'subject': 'Test'}
8         }],
9         'review_configs': [{
10            'action_name': 'send_email_tool',
11            'allowed_decisions': ['approve', 'edit', 'reject']
12        }]
13    }]
14 }
```

Listing 6: Middleware Interrupt State